Hibernate `ddl-auto`

The `spring.jpa.hibernate.ddl-auto` property controls how Hibernate manages the database schema based on the entity classes in a Spring Boot application.

1. `validate`

What does it do?

`validate` checks whether the database schema matches the entity classes.

It does not create, modify, or delete tables.

How does it work?

When the application starts, Hibernate compares the entities with the existing database tables and columns.

If everything matches, the application starts normally. If there is a mismatch, Hibernate throws an error.

Example

```
spring.jpa.hibernate.ddl-auto=validate
```

If we have:

```
@Entity
public class User {

    @Id
    private Long id;

    private String name;
}
```

Hibernate checks that the corresponding database table has the required columns.

Real-world Case

`validate` is commonly useful in production environments when the database schema is managed separately using tools such as Flyway or Liquibase.

Hibernate only verifies that the schema is correct.

---

2. `update`

What does it do?

`update` automatically updates the database schema to match the entity classes while keeping the existing data.

How does it work?

For example, if the entity contains:

```
@Entity
public class User {

    @Id
    private Long id;

    private String name;

    private String email;
}
```

If the `email` column does not exist in the database, Hibernate can add it automatically.

The existing data is not intentionally deleted when Hibernate performs these schema updates.

Example

```
spring.jpa.hibernate.ddl-auto=update
```

Real-world Case

`update` is commonly used during development because developers frequently change entity classes.

For example, when a new field is added to an entity, Hibernate can update the database automatically instead of requiring the developer to manually write an `ALTER TABLE` statement.

It is generally not recommended as the main schema-management strategy in production. Database migration tools such as Flyway or Liquibase are usually preferred.

---

3. `create`

What does it do?

`create` creates the database schema from the entity classes when the application starts.

The existing schema is dropped first, so existing data can be lost.

How does it work?

For example, if we have a `User` entity, Hibernate creates the corresponding `User` table when the application starts.

Conceptually, it works like:

```
DROP TABLE USERS;
CREATE TABLE USERS (...);
```

Example

```
spring.jpa.hibernate.ddl-auto=create
```

Real-world Case

`create` can be useful during development or testing when we want to start with a completely fresh database every time the application starts.

It should not normally be used in production because existing data can be deleted.

---

4. `create-drop`

What does it do?

`create-drop` creates the database schema when the application starts and drops the schema when the application stops.

How does it work?

When the application starts:

```
Application starts
       ↓
Hibernate creates the tables
       ↓
Application runs
```

When the application stops:

```
Application stops
       ↓
Hibernate drops the tables
```

Example

```
spring.jpa.hibernate.ddl-auto=create-drop
```

Real-world Case

This option is useful for testing, especially when we need a clean database for each test run.

It is not suitable for production because the tables are removed when the application stops.

---

5. `none`

What does it do?

`none` tells Hibernate not to manage the database schema.

Hibernate does not create, update, validate, or drop tables.

How does it work?

The database schema must already exist, and it is managed separately from Hibernate.

Example

```
spring.jpa.hibernate.ddl-auto=none
```

Real-world Case

`none` can be used in production when database changes are completely managed by another system, such as Flyway, Liquibase, or database scripts.

Hibernate simply works with the existing database structure.

Example: `update`

If we have:

```
spring.jpa.hibernate.ddl-auto=update
```

Hibernate checks the entity classes against the existing database schema.

For example:

```
@Entity
public class Product {

    @Id
    private Long id;

    private String name;

    private double price;
}
```

If the database table contains:

```
ID
NAME
```

but the entity also contains `price`, Hibernate can add the missing `PRICE` column.

So the database becomes:

```
ID
NAME
PRICE
```

Real-world Use Case

A common use case for `update` is a development environment.

During development, developers frequently add or modify fields in entity classes. Using `update` allows Hibernate to automatically apply many schema changes without manually writing SQL statements every time.

For production systems, schema migrations are usually handled with tools such as Flyway or Liquibase because they provide better control over database changes.

Comparison Table

Value	What does it do?	Real-world Case
`validate`	Checks that the database schema matches the entities without changing it.	Production when the schema is managed by Flyway or Liquibase.
`update`	Updates the existing database schema to match the entities while keeping existing data.	Development environments.
`create`	Drops the existing schema and creates a new one when the application starts.	Development and testing when a fresh database is needed.
`create-drop`	Creates the schema at startup and drops it when the application stops.	Testing environments.
`none`	Does not manage or validate the database schema.	Production when schema changes are managed separately.

Quick Summary

```
validate     → Check the schema only
update       → Update the existing schema
create       → Drop and create the schema
create-drop  → Create at startup and drop at shutdown
none         → Hibernate does nothing with the schema
```